

Lab 1: Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

```
mysql> create database schooldb;  
Query OK, 1 row affected (1.573 sec)
```

```
mysql> use schooldb;  
Database changed  
mysql> create table student(  
    -> student_id int primary key auto_increment,  
    -> student_name varchar(20) NOT NULL,  
    -> age INT NOT NULL,  
    -> class varchar(20),  
    -> address text);  
Query OK, 0 rows affected (1.275 sec)
```

- Lab 2: Insert five records into the students table and retrieve all records using the SELECT statement.

```
mysql> insert into student(student_name,age,class,address)  
    -> VALUES  
    -> ("yash",20,"12","ahmedabad"),  
    -> ("priya",17,"10","ahmedabad"),  
    -> ("kiya",16,"9","mumbai"),  
    -> ("harsh",17,"12","pune"),  
    -> ("bhargav",20,"12","vapi"),  
    -> ("mayank",19,"10","surat");  
Query OK, 6 rows affected (0.410 sec)  
Records: 6  Duplicates: 0  Warnings: 0
```

Lab 1: Write SQL queries to retrieve specific columns (student_name and age) from the students table.

```
mysql> select student_name , age from student;
+-----+-----+
| student_name | age |
+-----+-----+
| yash         | 20  |
| priya        | 17  |
| kiya         | 16  |
| harsh        | 17  |
| bhargav      | 20  |
| mayank       | 19  |
+-----+-----+
6 rows in set (0.018 sec)
```

- Lab 2: Write SQL queries to retrieve all students whose age is greater than 10.

```
mysql> select * from student where age>10;
+-----+-----+-----+-----+-----+
| student_id | student_name | age | class | address |
+-----+-----+-----+-----+-----+
| 1          | yash         | 20  | 12    | ahmedabad |
| 2          | priya        | 17  | 10    | ahmedabad |
| 3          | kiya         | 16  | 9     | mumbai    |
| 4          | harsh        | 17  | 12    | pune      |
| 5          | bhargav      | 20  | 12    | vapi      |
| 6          | mayank       | 19  | 10    | surat     |
+-----+-----+-----+-----+-----+
6 rows in set (0.038 sec)
```

- Lab 1: Create a table teachers with the following columns: teacher_id (Primary Key), teacher_name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

```
mysql> create table teacher(
-> teacher_id int primary key auto_increment,
-> teacher_name varchar(20) not null,
-> subject varchar(20) not null,
-> email varchar(30) unique );
Query OK, 0 rows affected (1.138 sec)
```

- Lab 2: Implement a FOREIGN KEY constraint to relate the teacher_id from the teachers table with the students table.

```
mysql> alter table teacher add student_id int references student(student_id);
Query OK, 0 rows affected (0.974 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

- Lab 1: Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key.

```
mysql> create table course(
  -> course_id int primary key auto_increment,
  -> course_name varchar(20) not null,
  -> course_credit int);
Query OK, 0 rows affected (0.428 sec)
```

- Lab 2: Use the CREATE command to create a database university_db.

```
mysql> create database universitydb;
Query OK, 1 row affected (0.125 sec)
```

Lab 1: Modify the courses table by adding a column course_duration using the ALTER command

```
mysql> alter table course add course_duration int ;
Query OK, 0 rows affected (1.091 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

- Lab 2: Drop the course_credits column from the courses table.

```
mysql> alter table course drop column course_credit;
Query OK, 0 rows affected (0.777 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Lab 1: Drop the teachers table from the school_db database.

```
mysql> drop table teacher;
Query OK, 0 rows affected (0.494 sec)
```

- Lab 2: Drop the students table from the school_db database and verify that the table has been removed.

```
mysql> drop table student;
Query OK, 0 rows affected (0.392 sec)
```

- Lab 1: Insert three records into the courses table using the INSERT command.

```
mysql> insert into course(
  -> course_name, course_duration) values
  -> ("python", 2),
  -> ("java", 3),
  -> ("php", 2),
  -> ("node.js", 4);
Query OK, 4 rows affected (0.134 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

- Lab 2: Update the course duration of a specific course using the UPDATE command.

```
mysql> update course set course_duration=5 where course_id=4;
Query OK, 1 row affected (0.088 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- Lab 3: Delete a course with a specific course_id from the courses table using the DELETE command.

```
mysql> delete from course where course_id=3;
Query OK, 1 row affected (0.130 sec)
```

- Lab 1: Retrieve all courses from the courses table using the SELECT statement.

```
mysql> select * from course;
+-----+-----+-----+
| course_id | course_name | course_duration |
+-----+-----+-----+
|          1 | python      |                4 |
|          2 | java        |                3 |
|          4 | node.js     |                5 |
+-----+-----+-----+
3 rows in set (0.015 sec)
```

- Lab 2: Sort the courses based on course_duration in descending order using ORDER BY.

```
mysql> select * from course order by course_duration desc;
+-----+-----+-----+
| course_id | course_name | course_duration |
+-----+-----+-----+
|          4 | node.js     |                5 |
|          1 | python      |                4 |
|          2 | java        |                3 |
+-----+-----+-----+
3 rows in set (0.098 sec)
```

- Lab 3: Limit the results of the SELECT query to show only the top two courses using LIMIT.

```
mysql> select * from course limit 2;
+-----+-----+-----+
| course_id | course_name | course_duration |
+-----+-----+-----+
|          1 | python      |                4 |
|          2 | java        |                3 |
+-----+-----+-----+
2 rows in set (0.014 sec)
```

- Lab 1: Create two new users user1 and user2 and grant user1 permission to SELECT from the courses table.

```
mysql> create user user1 identified by "bharat5369";
Query OK, 0 rows affected (0.258 sec)
```

```
mysql> create user user2 identified by "bharat5369";
Query OK, 0 rows affected (0.093 sec)
```

```
mysql> grant select
      -> on course
      -> to user1;
Query OK, 0 rows affected (0.105 sec)
```

- Lab 2: Revoke the INSERT permission from user1 and give it to user2.

```
mysql> select * from course;
ERROR 1142 (42000): SELECT command denied to user 'user1'@'localhost' for table 'course'
```

```
mysql> revoke select
      -> on course
      -> from user1;
Query OK, 0 rows affected (0.041 sec)
```

```
mysql> grant select on course
      -> to user2;
Query OK, 0 rows affected (0.057 sec)
```

- Lab 1: Insert a few rows into the courses table and use COMMIT to save the changes.

```
mysql> insert into course(course_id,course_name,course_duration) values
      -> (5,"php",4),
      -> (6,"html",4),
      -> (7,"css",5);
Query OK, 3 rows affected (0.103 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> commit;
Query OK, 0 rows affected (0.154 sec)
```

LAB EXERCISES

- Lab 1: Insert a few rows into the courses table and use COMMIT to save the changes.

```
mysql> alter table course add column course_credit varchar(10);
Query OK, 0 rows affected (0.808 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> commit;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'commit' at line 1
mysql> commit;
Query OK, 0 rows affected (0.010 sec)
```

- Lab 2: Insert additional rows, then use ROLLBACK to undo the last insert operation.

```
mysql> alter table course add column fees varchar(20);
Query OK, 0 rows affected (0.412 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> rollback;
Query OK, 0 rows affected (0.008 sec)
```

- Lab 3: Create a SAVEPOINT before updating the courses table, and use it to roll back specific changes.

```
mysql> start transaction;
Query OK, 0 rows affected (0.009 sec)

mysql> update course set course_name = "java script " where course_id=7;
Query OK, 1 row affected (0.026 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> savepoint s1;
Query OK, 0 rows affected (0.100 sec)

mysql> update course set course_name = " css " where id=7;
ERROR 1054 (42S22): Unknown column 'id' in 'where clause'
mysql> update course set course_name = " css " where course_id=7;
Query OK, 1 row affected (0.015 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> rollback to s1;
Query OK, 0 rows affected (0.016 sec)

mysql> select * from course;
```

course_id	course_name	course_duration	course_credit
1	python	4	NULL
2	java	3	NULL
4	node.js	5	NULL
5	php	4	NULL
6	html	4	NULL
7	java script	5	NULL

```
6 rows in set (0.012 sec)
```

- Lab 1: Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

```
mysql> select department.dept_id , dept_name, emp_name from department inner join employee on employee.dept_id=department.dept_id;
+-----+-----+-----+
| dept_id | dept_name | emp_name |
+-----+-----+-----+
| 1 | HR | amit |
| 2 | IT | yash |
| 3 | FINANCE | neha |
+-----+-----+-----+
3 rows in set (0.018 sec)
```

- Lab 2: Use a LEFT JOIN to show all departments, even those without employees.

```
mysql> select department.dept_id , dept_name , location from department left join employee on department.dept_id=employee.dept_id;
+-----+-----+-----+
| dept_id | dept_name | location |
+-----+-----+-----+
| 1 | HR | ahmedabad |
| 2 | IT | gandhinager |
| 3 | FINANCE | surat |
+-----+-----+-----+
3 rows in set (0.020 sec)
```

- Lab 1: Group employees by department and count the number of employees in each department using GROUP BY.

```
mysql> select dept_name , department.dept_id, count(department.dept_id) from department inner join employee on employee.dept_id = employee.dept_id group by department.dept_id;
+-----+-----+-----+
| dept_name | dept_id | count(department.dept_id) |
+-----+-----+-----+
| HR | 1 | 3 |
| IT | 2 | 3 |
| FINANCE | 3 | 3 |
+-----+-----+-----+
3 rows in set (0.017 sec)
```

- Lab 2: Use the AVG aggregate function to find the average salary of employees in each department.

```
mysql> select avg(salary) from employee;
+-----+
| avg(salary) |
+-----+
| 26000.0000 |
+-----+
1 row in set (0.056 sec)
```

- Lab 1: Create a view to show all employees along with their department names.

```
mysql> create view emp as select emp_id , emp_name , salary from employee inner join department on department.dept_id=employee.dept_id;
Query OK, 0 rows affected (0.271 sec)

mysql> select * from emp;
+-----+-----+-----+
| emp_id | emp_name | salary |
+-----+-----+-----+
| 101 | yash | 30000 |
| 102 | amit | 20000 |
| 103 | neha | 28000 |
+-----+-----+-----+
3 rows in set (0.146 sec)
```

- Lab 2: Modify the view to exclude employees whose salaries are below \$50,000.

```
mysql> create view salary as select * from employee where salary < 50000;
Query OK, 0 rows affected (0.209 sec)

mysql> select * from salary ;
+-----+-----+-----+-----+
| emp_id | emp_name | salary | dept_id |
+-----+-----+-----+-----+
| 101    | yash     | 30000  | 2       |
| 102    | amit     | 20000  | 1       |
| 103    | neha     | 28000  | 3       |
+-----+-----+-----+-----+
3 rows in set (0.030 sec)
```

- Lab 1: Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint.

```
mysql> start transaction
-> ;
Query OK, 0 rows affected (0.008 sec)

mysql> insert into employee values
-> (104,"rohan",10000,4);
Query OK, 1 row affected (0.014 sec)

mysql> insert into employee values
-> (104,"rohan",10000,4)^C
mysql> savepoint s1;
Query OK, 0 rows affected (0.008 sec)

mysql> insert into employee values
-> (105,"siya",20000,4),
-> (106,"diya",18000,5);
Query OK, 2 rows affected (0.012 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> rollback to s1;
Query OK, 0 rows affected (0.011 sec)

mysql> select * from employee;
+-----+-----+-----+-----+
| emp_id | emp_name | salary | dept_id |
+-----+-----+-----+-----+
| 101    | yash     | 30000  | 2       |
| 102    | amit     | 20000  | 1       |
| 103    | neha     | 28000  | 3       |
| 104    | rohan    | 10000  | 4       |
+-----+-----+-----+-----+
4 rows in set (0.012 sec)
```

- Lab 2: Commit part of a transaction after using a savepoint and then rollback the remaining changes.


```

mysql> start transaction;
Query OK, 0 rows affected (0.125 sec)

mysql> update employee set salary = 12000 where emp_id=104;
Query OK, 1 row affected (0.034 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> savepoint s1;
Query OK, 0 rows affected (0.009 sec)

mysql> update employee set salary = 25000 where emp_id=103;
Query OK, 1 row affected (0.014 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> commit;
Query OK, 0 rows affected (0.122 sec)

mysql> rollback to s1;
ERROR 1305 (42000): SAVEPOINT s1 does not exist
mysql> rollback
-> ;
Query OK, 0 rows affected (0.008 sec)

mysql> select * from employee;
+-----+-----+-----+-----+
| emp_id | emp_name | salary | dept_id |
+-----+-----+-----+-----+
| 101 | yash | 30000 | 2 |
| 102 | amit | 20000 | 1 |
| 103 | neha | 25000 | 3 |
| 104 | rohan | 12000 | 4 |
+-----+-----+-----+-----+
4 rows in set (0.013 sec)

```